

Bags, Vectors & Transformers

Contextual Embeddings & Transformers · Day 3

Denise J. Roth, PhD Candidate

Strategic Communication Group · Wageningen University & Research

Where We Have Been

Two days, one journey — from counting words to representing meaning:

Day 1: Bags

- Words as counts
- Document-term matrix
- TF-IDF, dictionaries, topics
- Simple, transparent

Day 2: Vectors

- Meaning from context
- Word2Vec, GloVe
- Dense embeddings
- **But: one vector per word**

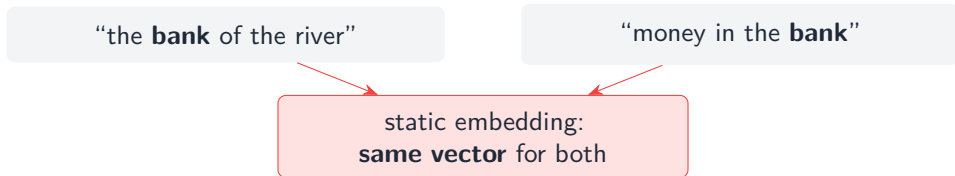
Day 3: Transformers

- Meaning from *this* context
- Attention
- BERT & friends
- **A vector per use**

Today we close the loop: the context problem that static embeddings could not solve.

The Problem, One More Time

Static embeddings give each word **one fixed vector**. But words change meaning with context — and a single vector cannot be in two places at once.



We want representations that **read the sentence** and give “bank” a *different* vector depending on whether it is about water or money. That is what **contextual embeddings** do.

What Contextual Embeddings Unlock

New capabilities

- **Word sense** disambiguation (“bank”, “spring”)
- Represent **whole sentences** richly
- Capture **syntax & negation** (“not good” $\neq$ “good”)
- Strong **transfer**: fine-tune for your task with little data

What this means for you

- State-of-the-art **classification**
- Better **sentiment** / **stance** on hard cases
- **Semantic search** across documents
- A foundation for the **LLMs** everyone is talking about

The jump from static to contextual is what took NLP from “useful” to “everywhere” over the last few years.

The Breakthrough: “Attention Is All You Need”

“Attention Is All You Need”

Vaswani et al., 2017 — the paper that introduced the **transformer**

Before 2017, models read text **word by word, in order** (recurrent networks) — slow, and forgetful over long distances. The transformer threw that out and replaced it with one mechanism: **attention**, which lets every word look at every other word **at once**.

That single idea powers **BERT**, **GPT**, and essentially every modern language model. The next few slides build up what “attention” actually means.

Attention: The Core Intuition

To understand a word, you look at the **other words that give it meaning**. Attention makes a model do exactly this — for every word, it decides **how much to focus on each other word** in the sentence.



To represent **“it”**, the model pays **strong attention** to **“animal”** (that is what “it” refers to) and a little to “tired”. The thickness of each arrow is the **attention weight** — how relevant that word is.

Crucially, the model learns these weights from data — nobody hand-codes that “it” means “animal” here.

How Attention Works — Without the Math

For each word, attention runs a little **matching process**. A useful analogy is **searching**:

Query

What *this* word is looking for.

“**it**”: what do I refer to?

Key

What each *other* word offers.

“animal”: I’m a noun, a thing

Value

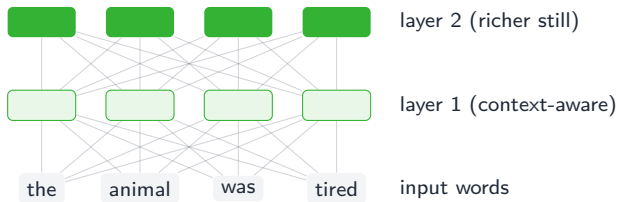
The information actually *passed along* when there’s a match.

The word’s **query** is compared with every other word’s **key**. Good matches get high **attention weights**, and the matching words’ **values** are blended together to form the word’s new, **context-aware** representation.

Query–key–value: the whole transformer is built by stacking this one operation many times.

From One Layer to a Deep Model

Attention runs **in parallel** for every word (this is **self-attention** — the sentence attends to itself). Then we **stack many layers**, each refining the representations further.



Each layer lets every word gather information from all the others. After a dozen layers, each word's vector reflects the **entire sentence**. **BERT** has 12 such layers; large models have many more.

How BERT Learns: Masked Language Modeling

A transformer needs to be **trained** on something. **BERT** — *Bidirectional Encoder Representations from Transformers* (Devlin et al., 2018) — uses a clever self-supervised task: **fill in the blank**.

“the animal didn’t cross because [MASK] was tired”



predict the hidden word: “it”

Hide 15% of words and train the model to **predict them from context**. To do this well, it *must* learn grammar, meaning, and world knowledge — all **without human labels**.

The “**Bidirectional**” in the name matters here: to guess the blank, BERT reads the words *both before and after* it — unlike left-to-right models that only look back. (*The same “fake task” trick as Word2Vec, but far more powerful.*)

Two Families: Encoders and Decoders

Encoders (BERT-style)

- Read the **whole sentence at once**
- Built for **understanding** & analysis
- Great for **classification**, search, feature extraction
- **Our focus** — best fit for social-science analysis

Decoders (GPT-style)

- Read **left to right**, predict the next word
- Built for **generation**
- Power **ChatGPT** and the LLMs

Same attention engine, different objective.

For measuring and classifying text — what social scientists usually want — encoders like BERT are typically the right tool.

Mechanics: The Takeaways

- ✓ **Contextual embeddings** give a word a **different vector in each sentence** — solving the “bank” problem
- ✓ They are built on the **transformer** (“Attention Is All You Need”, 2017)
- ✓ **Attention** lets each word focus on the other words that matter — via **query, key, value**
- ✓ **BERT** learns by **masked language modeling**: fill in the blank, no labels needed
- ✓ **Encoders** (BERT) are for understanding; **decoders** (GPT) are for generation

Next: we put a real BERT model to work in Python — contextual vectors, the “bank” test, and text classification.